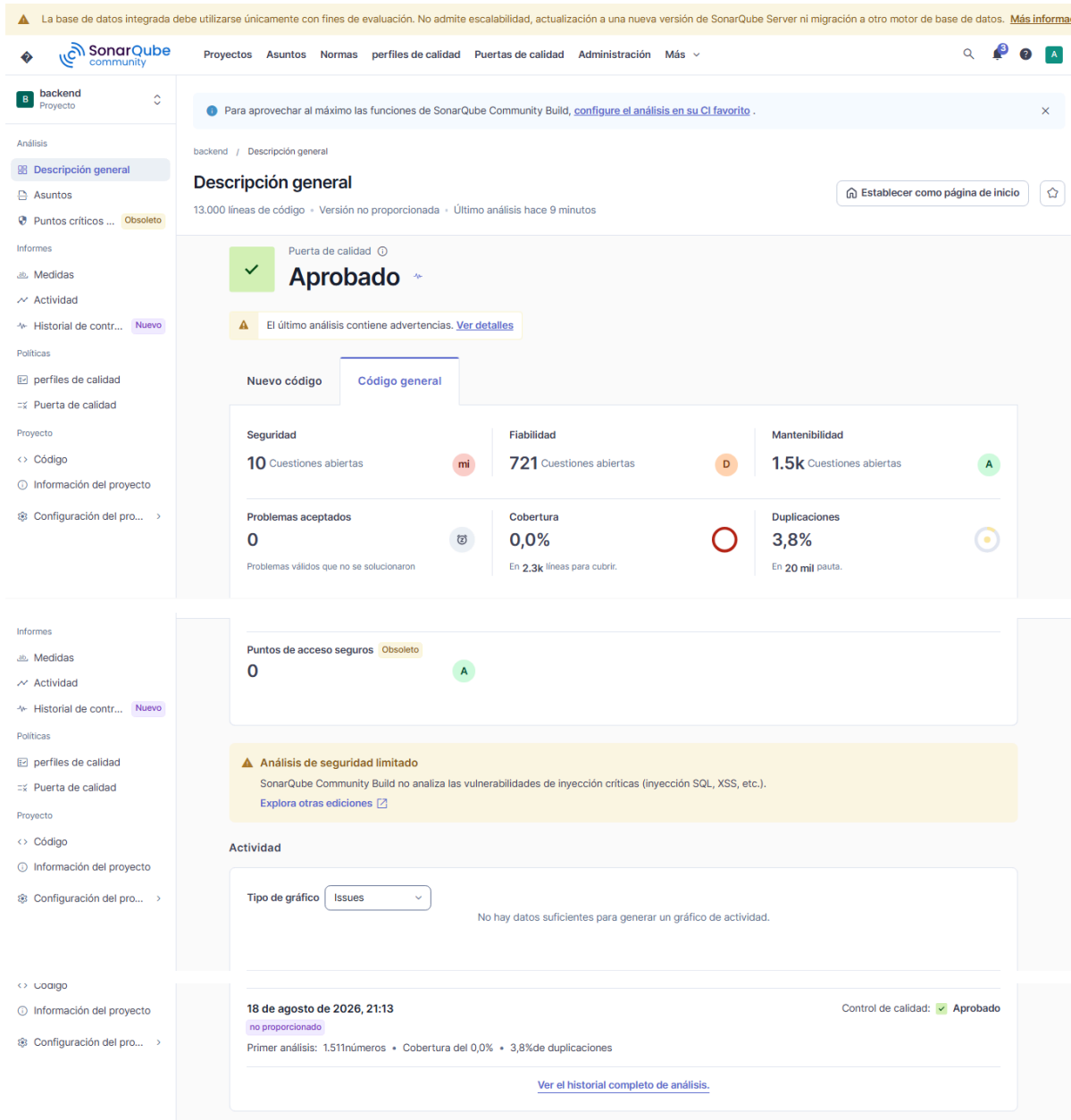


Informe sobre la calidad del código

18/08/2026



Lo más urgente: se encontró que hay una clave secreta (la credencial de acceso al servicio de Firebase) guardada directamente dentro del código, algo que nunca debería pasar porque cualquiera con acceso al repositorio podría usarla.

Además, la herramienta indica que el proyecto tiene 0% de pruebas cubiertas, pero al revisar los registros de ejecución de pruebas, se confirma que sí existen pruebas y que la gran mayoría funciona correctamente (más del 90%). El 0% que muestra SonarQube es simplemente porque no se le pasó la información de esas pruebas, no porque no existan.

General

	Problemas encontrados	Estado	Tiempo estimado para arreglarlo
Seguridad	10	Delicado (hay 1 dato sensible expuesto)	2 h 30 min
Estabilidad del sistema	721	Necesita atención (nota D)	7 días 3 h
Orden y limpieza del código	1.464	Bien (nota A)	14 días
Pruebas automáticas	SonarQube marca 0,0%	Dato engañoso, ver punto 6	—
Código repetido	3,8% del código	Aceptable	—

Seguridad

Se encontraron 10 puntos relacionados con seguridad. Arreglarlos tomaría, en total, unas 2 horas y media.

Nivel de urgencia	Cuántos
Urgente	1
Importante	2
Medio	4
Menor	3

Lo más importante

- **Urgente:** hay una clave de acceso a Firebase guardada dentro de un archivo del proyecto. Se debe cambiar esa clave cuanto antes y quitarla del código, para que nadie no autorizado pueda usarla.
- **Importante:** la forma en que se arma una de las piezas del proyecto (relacionada con una librería externa llamada bcrypt) copia archivos de más, lo que podría filtrar información sin querer. Se recomienda ajustar ese proceso de construcción.
- **Medio:** en varias partes del sistema (pedidos personalizados, pedidos normales y usuarios) se usa una forma de generar números al azar que no es lo suficientemente segura para casos sensibles. Convendría usar un método más robusto.
- **Menor:** una parte del sistema todavía usa una conexión sin cifrar (HTTP en lugar de HTTPS), lo cual es menos seguro.

Estabilidad del sistema (posibles errores en tiempo de ejecución)

Se encontraron 721 puntos que podrían generar comportamientos inesperados o errores mientras el sistema está funcionando. En conjunto, corregirlos tomaría aproximadamente 7 días de trabajo.

Nivel de urgencia	Cuántos
Urgente	0
Importante	1

Medio	46
Menor	674

La mayoría de estos puntos (674) son de baja gravedad, pero como son muchos, suman bastante tiempo de trabajo y por eso el sistema recibe una nota baja (D) en este aspecto. No es una señal de que el sistema esté fallando gravemente hoy, sino de que hay mucho "mantenimiento pendiente" acumulado.

Orden y facilidad para mantener el código

Se encontraron 1.464 puntos relacionados con qué tan fácil (o difícil) es entender y modificar el código a futuro. En total tomaría unos 14 días arreglarlos todos, aunque la nota general es buena (A).

Nivel de urgencia	Cuántos
Urgente	8
Importante	9
Medio	709
Menor	734
Informativo	4

Aunque la nota general es buena, es el área con más trabajo acumulado en total, y tiene 8 puntos urgentes y 9 importantes que conviene priorizar antes que el resto.

Sobre las pruebas automáticas (por qué SonarQube marca 0%)

SonarQube muestra que el proyecto tiene 0% de "cobertura de pruebas", es decir, según esa herramienta no habría nada probado automáticamente. Sin embargo, al revisar los registros de ejecución de pruebas que compartiste, queda claro que esto no es así: el proyecto sí tiene una cantidad importante de pruebas automáticas, y la mayoría de ellas pasa sin problema.

Tipo de prueba	Pasaron	Fallaron	% de éxito
Pruebas unitarias	106 de 112	4	95%
Pruebas de flujo completo	91 de 107	5	85%

En otras palabras: el 0% no significa que nadie haya escrito pruebas, sino que esos resultados de pruebas nunca se le avisaron a SonarQube. Es un tema de configuración pendiente, no un problema de que falte trabajo de pruebas desde cero.

Lo que sí funciona bien

- La mayoría de las pruebas pasa correctamente: 106 de 112 pruebas unitarias (95%) y 91 de 107 pruebas de flujo completo (85%), cubriendo áreas como usuarios, pedidos, movimientos de inventario, notificaciones, carrito de compras y pagos.

Por qué fallan las demás

- Algunas pruebas ya no encuentran el archivo que deberían probar, porque el código se reorganizó y esas pruebas quedaron desactualizadas (apuntan a una ubicación antigua).
- Algunos archivos de prueba están vacíos, es decir, existen pero todavía no se escribió ninguna prueba dentro de ellos.
- En un par de casos, las pruebas usan una versión "simulada" del sistema que no está del todo actualizada respecto al código real, así que fallan por esa desincronización y no porque el sistema esté roto.
- También se detectaron algunos conflictos con los datos de prueba (por ejemplo, al intentar borrar un registro que todavía está siendo usado por otro), lo cual sugiere revisar el orden en que se crean y eliminan los datos durante las pruebas.
- Y hay dos casos donde sí conviene confirmar si es un error real del sistema: al crear un producto con un nombre repetido, o al subir una imagen válida, la respuesta del sistema no es la que se esperaba.

Código repetido

Un 3,8% del código está duplicado en distintas partes del proyecto, lo cual es un nivel razonable y no urgente.

- El archivo con más repetición es uno de utilidades de pruebas (auth-helper.ts), con más de la mitad de sus líneas repetidas en otro lugar.
- También hay repetición en archivos que pertenecen a librerías externas (no código propio), así que esa parte no requiere ninguna acción.

Qué hacer primero (Por prioridad)

- **1. Cambiar de inmediato la clave de acceso a Firebase que quedó expuesta en el código, y asegurarse de que no vuelva a subirse por accidente.**
- 2. Resolver los 17 puntos marcados como urgentes o importantes en la sección de mantenibilidad (8 urgentes + 9 importantes), antes de seguir agregando funciones nuevas.
- 3. Conectar los resultados de las pruebas automáticas con SonarQube, para que deje de mostrar 0% de forma incorrecta. De paso, actualizar las pruebas que quedaron desactualizadas, completar las que están vacías y revisar los dos casos donde el sistema no respondió como se esperaba.
- 4. Reemplazar la forma de generar números al azar en los puntos sensibles identificados, por una opción más segura.
- 5. Ajustar el proceso de construcción de la aplicación para que no copie archivos de más, y activar la conexión segura (HTTPS) donde todavía falta.
- 6. Reducir la repetición de código en el archivo de utilidades de pruebas mencionado.